Adapting Large Language Models for Encrypted Traffic Analysis Services: An Efficient Realization With Mixture of LoRA Experts

Yi Liu, Xiang Zheng, Chengjun Cai, *Member, IEEE*, Xingliang Yuan, *Senior Member, IEEE*, and Cong Wang, *Fellow, IEEE*

***Abstract*—As encrypted traffic grows, traditional rule-based and deep learning methods struggle with engineering costs and encryption complexity. While Large Language Models (LLMs) offer promise for traffic analysis via pre-trained feature learning, they face challenges in handling diverse tasks, retaining pre-training knowledge, and adapting efficiently. To address these issues, we propose a new traffic representation learning method and a new Parameter-Efficient Fine-Tuning (PEFT) method for multi-task encrypted traffic analysis services, called `TrafficLLM`. `TrafficLLM` alleviates task heterogeneity by utilizing a universal multi-task prompt template and addresses pre-training knowledge forgetting by integrating Singular Value Decomposition based Low-Rank Adaptation (SVD-LoRA). To further reduce the cost of adapting to multiple tasks, we combine the strengths of the Mixture of Experts (MoE) for multi-task learning with SVD-LoRA for PEFT, enabling efficient multi-task traffic analysis. Additionally, we introduce task-aware gating functions to dynamically assign different weights to experts, facilitating the efficient fusion of expert knowledge. Comprehensive experiments on 7 datasets across 5 downstream tasks demonstrate that `TrafficLLM` delivers superior analysis performance and resource efficiency compared to state-of-the-art models, including DeepSeek, NetGPT, ET-BERT, and TFE-GNN. Detailed analysis of throughput, memory usage, and latency further highlights the practical advantages of `TrafficLLM`.**

***Index Terms*—Encrypted traffic classification, LLMs, LoRA, mixture of experts.**

Received 9 July 2025; revised 15 January 2026; accepted 23 February 2026. Date of publication 9 March 2026; date of current version 10 April 2026. This work was supported in part by the Hong Kong Research Grants Council under Grant CityU 11218322, Grant 11219524, Grant R6021-20F, Grant R1012-21, Grant RFS21221S04, Grant C2004-21G, Grant C1029-22G, Grant C6015-23G, and Grant N_CityU139/21, and in part by the Innovation and Technology Commission (ITC) under the Joint Mainland-Hong Kong Funding Scheme (MHKJFS) under Grant MHP/135/23. This work was also supported in part by the InnoHK initiative, the Government of the HKSAR, in part by the Laboratory for AI-Powered Financial Technologies (AIFT), and in part by the Guangdong and Hong Kong Universities "1+1+1" Joint Research Collaboration Scheme. *(Corresponding author: Cong Wang.)*

Yi Liu, Xiang Zheng, and Cong Wang are with the Department of Computer Science, City University of Hong Kong, Kowloon Tong Hong Kong (e-mail: yiliu247@cityu.edu.hk; xzheng235-c@my.cityu.edu.hk; congwang@cityu.edu.hk).

Chengjun Cai is with the Computer Science and Information Technology Center, City University of Hong Kong (Dongguan), Dongguan 523808, China (e-mail: chengjun.cai@cityu-dg.edu.cn).

Xingliang Yuan is with the School of Computing and Information Systems, University of Melbourne, Parkville, VIC 3010, Australia (e-mail: xingliang.yuan@unimelb.edu.au).

Our data and code are available at https://github.com/yiliucs/TrafficLLM.

Digital Object Identifier 10.1109/TSC.2026.3671484

I. Introduction

ENCRYPTED network traffic analysis has become a critical component of network security and management due to the widespread adoption of encryption protocols such as HTTPS, which protect sensitive data during online communications [1]. Broadly, encrypted traffic analysis services aim to develop advanced algorithms and tools (e.g., Cisco Encrypted Traffic Analytics) that can accurately identify and classify encrypted traffic without compromising user privacy [2]. This capability enables organizations to infer accessed applications or services within their networks, such as mobile app classification [3] or service identification [4]. To this end, various traditional methods have been proposed, including port-based approaches [5], payload or packet inspection-based techniques [6], and traffic statistics feature-based methods [7]. However, these approaches rely heavily on *rule engineering*, requiring substantial manual effort to design and maintain classification rules [8]. Moreover, the rapid evolution of encryption technologies has increasingly limited their adaptability to emerging encryption schemes [9]. As a result, there is an urgent need for more generalizable methods that can capture implicit and robust patterns in encrypted traffic, enabling accurate and universal traffic analysis [8], [9], [10].

To eliminate the need for rule engineering, recent advances in Deep Learning (DL) have been applied to encrypted traffic classification [11], [12], [13]. DL techniques, particularly neural network architectures such as Convolutional Neural Networks (CNNs) [14] and Graph Neural Networks (GNNs) [12], are capable of capturing complex patterns in encrypted traffic, enabling more accurate and adaptive analysis. Early studies typically employed a single DL model trained on large-scale labeled datasets to perform a specific task, such as protocol identification [15] or application classification [16]. As network management tasks become more complex, multi-task DL models that leverage multi-task learning [17] have been proposed to simultaneously address multiple encrypted traffic analysis tasks [18]. Overall, DL-based approaches enable network management systems to better adapt to evolving encryption schemes and substantially outperform hand-crafted rule-based methods.

Despite their promising potential, existing DL-based algorithms still suffer from two key limitations (see Fig. 1): *1) High Model Engineering Costs.* The focus of DL-based algorithms

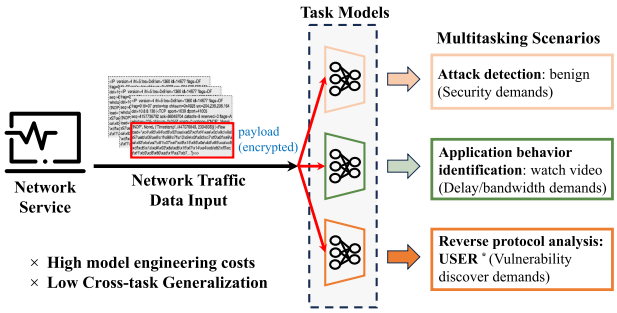


Fig. 1. Multi-task scenarios for DL-based traffic data analysis. Since the above-mentioned heterogeneous tasks require different specific models, DL-based methods suffer from high model engineering costs and low cross-task generalization capabilities.

has shifted from rule engineering to model engineering [19], with success largely dependent on manually tailoring DL models for specific encrypted traffic analysis tasks. This process is labor-intensive due to the complexity of DL models. Additionally, the diversity of tasks prevents the use of a single DL model across different tasks, necessitating specialized models for each task [8]. This "one model for one task" approach further escalates engineering costs. *2) Low Generalization.* Current DL models are typically tailored to specific data distributions or environments, overlooking commonalities across diverse encrypted traffic behaviors [9]. This leads to poorer performance on unseen data distributions compared to rule-based algorithms. Although transfer learning can address these challenges, its effectiveness depends on access to large-scale annotated traffic data. As a result, the generalization ability across different tasks is weakened, hindering the widespread deployment of DL-based algorithms in practice.

The advent of pre-training techniques [20] and Large Language Models (LLMs; e.g., GPT-2 [21]) built upon the Transformer architecture [22] offers promising solutions to the aforementioned challenges. Through large-scale pre-training, LLMs exhibit strong capabilities in knowledge transfer and representation learning, demonstrating superior performance in feature extraction, few-shot learning, and multi-task learning [23], [24], [25], [26]. As a result, they have been widely adopted in natural language processing, computer vision, and increasingly in cybersecurity research [27], [28], [29]. In particular, LLMs pre-trained for traffic analysis [30], [31], [32], such as NetGPT [8] and ET-BERT [9], have achieved remarkable performance. These models can accurately align traffic inputs with corresponding analysis outputs for specific tasks, e.g., encrypted malicious traffic classification [8]. This progress naturally raises a key question: *can we fully embrace the era of LLMs and efficiently adapt them to solve diverse traffic analysis tasks in a unified manner?*

Despite the considerable enhancement in representation learning that LLMs bring to downstream tasks, e.g., mobile application classification, attack detection, and traffic behavior classification, several challenges persist due to the distinct requirements and characteristics of network traffic and tasks.

- *C1: Heterogeneous Task Alignment:* On one hand, encrypted traffic classification involves diverse traffic types (e.g., HTTP, DNS, and SMTP) and heterogeneous subtasks, such as mobile application classification and behavior analysis [16]. The need to extract heterogeneous headers and payloads from multimodal network traffic therefore poses a major challenge. Moreover, advances in encryption technologies increasingly reduce the availability of shared extractable traffic features [8], further exacerbating the difficulty of accurate heterogeneous task alignment.
- *C2: Pre-training Knowledge Forgetting:* In general, pre-trained LLMs typically require fine-tuning [8], [26], [33], a training technique tailored to new tasks, before they can effectively address encrypted traffic analysis tasks. However, existing methods appear to prioritize achieving accurate analysis while overlooking the considerable pre-training knowledge (e.g., common sense and traffic patterns) associated with LLMs [8]. In this context, prolonged fine-tuning may lead to the phenomenon of catastrophic forgetting, where the model loses previously acquired knowledge [34].
- *C3: High Adaptation Costs:* Adaptation costs in multi-task traffic analysis include fine-tuning, training, and inference overheads. Existing LLMs, such as ET-BERT [9], require task- and dataset-specific fine-tuning, resulting in high adaptation costs, while approaches like NetGPT [8] rely on large-scale model training, incurring substantial training overhead. Moreover, the use of extensive parameters to enhance feature extraction often degrades inference efficiency, introducing latency in network management systems [14]. The high memory demands of instruction processing further limit throughput, largely due to the attention mechanism inherent to LLMs.

Design Goals: The aforementioned challenges call for a holistic solution toward efficient LLMs for encrypted traffic classification, which is characterized by the following essential design goals: (G1) Robust Performance: The designed LLMs can achieve precise task alignment for multiple and multi-source encrypted traffic data. (G2) Strong Versatility: LLMs retain pre-training knowledge while efficiently completing traffic analysis tasks. (G3) Energy Efficient: Adaptation cost should be acceptable. In this paper, we formulate an input-agnostic service for encrypted traffic analysis, leveraging a singular LLM (e.g., ChatGLM [37]) to accomplish diverse and refined encrypted traffic data analysis tasks.

Our Contributions: To achieve the above design goals, we propose `TrafficLLM`, an LLM service specially formulated for encrypted traffic analysis, which can effectively adapt LLMs to multiple encrypted traffic analysis tasks while maintaining efficient inference. `TrafficLLM` comprises three core components: traffic representation learning, an efficient low-rank adapter, and a mixture of expert networks. Specifically, we extract the key five-tuple and statistical features from the encrypted traffic data of heterogeneous multi-tasks, leveraging a designed multi-task prompt template to normalize this data into text input for effective processing by `TrafficLLM` (**G1**). Building on this, we design a singular value decomposition-aware low-rank adapter (SVD-LoRA) to retain pretrain knowledge (**G2**) and reduce adaptation costs (**G3**). Additionally, we develop a

mixture of LoRA expert networks to enable TrafficLLM to adapt to multiple heterogeneous tasks seamlessly (**G1**). Extensive case studies on 7 large-scale encrypted traffic datasets, encompassing 5 distinct traffic classification tasks, demonstrate that TrafficLLM attains state-of-the-art performance, with an average accuracy of 95.76%, while offering superior throughput-latency trade-offs, achieving low adaptation costs. In addition, we deployed the quantized version (INT4) of TrafficLLM in an A100 environment and evaluated its performance on the open-world NUDT-Mobile [38] dataset. The results demonstrate that TrafficLLM achieves an F1 score of 0.9709, with a throughput of 800 samples per second and a latency of 400 milliseconds per sample, highlighting its effectiveness and efficiency in real-world scenarios.

The contributions of our work can be summarized as follows:

- We customize a multi-task LLM service for encrypted traffic data analysis, named TrafficLLM, which efficiently implements multi-task traffic analysis while maintaining low adaptation cost.
- We design an SVD-driven LoRA (SVD-LoRA) fine-tuning method to maximize the retention of pre-training knowledge and efficiently adapt to new tasks.
- We develop a MoE-enabled SVD-LoRA multi-task learning method to efficiently handle traffic tasks by integrating multiple expert knowledge. In addition, to further improve the performance, we design a task-aware gating to configure different contribution weights for specific tasks.
- We implement our design and conduct extensive experiments on 7 datasets related to 5 traffic downstream tasks to explore the performance, efficiency, generality, and parameter sensitivity of TrafficLLM. Compared with the baselines, our method can achieve 95.76% average accuracy on all tasks while maintaining low latency.

II. Related Work

Traffic Representation Learning: With the increasing volume of network traffic and the rapid evolution of encryption protocols, there is an urgent need for general traffic analysis tools for encrypted traffic data [31]. In this context, traffic representation learning [8], [9], [18] aims to extract input-agnostic general traffic features to enable robust encrypted traffic analysis. Traditional machine learning methods [5], [6] typically extract statistical features at the packet or flow level, such as packet size distribution or arrival time intervals, to represent traffic for accurate analysis. However, these features often suffer from over-compression, leading to the loss of crucial information inherent in the original datagram [11]. This can cause model degradation or even failure, particularly in complex multi-task traffic analysis scenarios. To address this issue, DL-based methods [12], [13] aim to utilize raw traffic bytes to extract more comprehensive traffic representations. However, these methods encounter problems such as ignoring IP headers, retaining non-anonymous biases, overlooking byte balance, or using improper data splitting techniques [39].

Large Models for Encrypted Traffic Data Analysis: With the rise of encryption protocols such as TLS/SSL, there is a growing trend to use large-scale DL models with powerful feature learning capabilities, surpassing traditional methods [26], [35], [36]. One notable line of research includes the utilization of complex DL architectures, such as CNNs and GNNs [36] with LSTM units, to discern complex patterns in unencrypted packet payloads. Studies have demonstrated the effectiveness of these models in classifying encrypted traffic using features like flow statistics and packet headers [26], [35], [36]. Another approach involves using transformer-based models like BERT [9] or its variants for time-series analysis. Researchers have adapted these models (e.g., NetGPT [8] and ET-BERT [9]) to work with sequences of packet metadata to classify encrypted traffic accurately. These large-scale pre-trained models have shown remarkable success in natural language processing tasks and have translated their effectiveness to network traffic analysis.

Multitask Learning in LLMs: Multi-task Learning (MTL) in LLMs enhances efficiency and performance by training on multiple related tasks simultaneously, leveraging shared representations and knowledge transfer [8], [40]. Effective MTL requires strategic task weighting, curriculum learning, and meta-learning to balance contributions across tasks [41]. These approaches incorporate task-specific layers or heads to address unique task requirements while benefiting from shared underlying structures. For example, NetGPT [8] designs a unique task identifier to be incorporated into the pre-training and fine-tuning process to achieve efficient multi-task learning. Then, the MTL techniques in existing LLms fail to effectively approach the high adaptation cost problem. We summarize the differences and connections between existing work and TrafficLLM in Table I.

III. Problem Definition

LLMs for Encrypted Traffic Analysis Services: We give a formal problem definition of encrypted traffic analysis services in the context of LLMs. Given a large corpus of encrypted traffic data, the task involves utilizing LLMs to simultaneously address multiple sub-tasks (e.g., malicious traffic detection and traffic behavior classification) related to encrypted traffic data analysis. Let $\mathcal{D} = \{D_1, D_2, \ldots, D_m\}$ represent the dataset, where each D_i corresponds to a specific task scenario or domain of encrypted network traffic. Within each dataset D_i, the network traffic is represented as sequences of tokens, where each token represents a component of the traffic data (e.g., packet headers, payload). In contrast to prior work [9] that applies large models to single-task traffic analysis via fine-tuning, our goal is to develop a multi-task LLM that jointly models multiple aspects of encrypted traffic across diverse scenarios. This problem is inherently cross-scenario, as effective analysis requires understanding encrypted communications under varying network environments, applications, and user behaviors.

The designed model needs to generalize well across diverse scenarios and adapt to new or unseen encrypted traffic patterns. Thus, let $\mathcal{T} = \{T_1, T_2, \ldots, T_k\}$ represent the set of traffic analysis tasks, where each T_j corresponds to a specific aspect of encrypted traffic analysis (e.g., encryption algorithm, protocol). The LLM is trained (or fine-tuned) to predict the labels for all tasks T_j simultaneously, leveraging shared representations across tasks to improve generalization. The objective is to learn a multi-tasking LLM $f : X \rightarrow Y_1 \times Y_2 \times \cdots \times Y_k$, where X

TABLE I
COMPARISON BETWEEN EXISTING WORK AND TrafficLLM. NOTE THAT SINCE DL-BASED METHODS DO NOT INVOLVE FINE-TUNING, THEY ARE NOT APPLICABLE IN TERMS OF FINE-TUNING METHODS AND RETAINING PRE-TRAINING KNOWLEDGE.

Method	Traffic Representation					Mtlti-Task?	Fine-Tuning	Retain Knowledge?	Adaption Cost	# of Tasks
	Header	Payload	IP Anonymizing	Data Balance	Splitting					
FS-Net [35]	✓	✓	✗	✗	Patch	✗	NA	NA	Low	1
AppScanner [3]	✓	✓	✓	✗	Patch	✗	NA	NA	Low	1
FlowPic [12]	✓	✓	✗	✗	Patch	✗	NA	NA	High	1
PERT [33]	✗	✓	✗	✗	Token	✗	NA	NA	High	1
TFE-GNN [36]	✓	✓	✗	✗	Token	✗	NA	NA	High	1
ET-BERT [9]	✗	✓	✗	✗	Token	✗	Full Parameter Tuning	✗	High	1
NetGPT [8]	✓	✓	✗	✗	Token	✓	Full Parameter Tuning	✗	High	5
Ours	✓	✓	✓	✓	Token	✓	SVD-LoRA	✓	Low	5

represents the input sequence of tokens and Y_j represents the set of possible labels for task T_j. Mathematically, the above process can be represented as:

$$\theta^* = \arg\min_{\theta} \sum_{i=1}^{m} \sum_{j=1}^{k} L\left(f(D_i;\theta)_{T_j}, y_i^{(T_j)}\right), \tag{1}$$

where L is the loss function (e.g., cross entropy loss), and $y_i^{(T_j)}$ denotes the ground truth labels for task T_j in dataset D_i.

Example: In practice, to accurately perform encrypted traffic analysis tasks, we format the input features as prompts for the LLMs, constructing instruction-answer pairs for fine-tuning, a process known as instruction tuning. Specifically, the input instruction I includes extracted traffic features, task information, and task labels. The LLMs then make predictions based on these instructions and output labels (i.e., the answer A). We subsequently calculate evaluation metrics such as precision and recall for specific tasks based on the LLMs' outputs. Below, we provide an example to demonstrate this concept.

Prompt Template Example

User inputs the prompt: I = "Input: {"packet_size_mean": 512, "packet_size_std": 10, "inter_arrival_time_mean": 0.02, "burst_size_mean": 100, "flow_duration": 60.} Task: Classify the above encrypted traffic flow into one of the following categories: Web Browsing, Video Streaming, File Transfer, Potential Malicious Activity."

LLM makes the prediction: A = "Web Browsing".

Existing LLMs and Their Limitations: LLMs like NetGPT [8] exhibit notable limitations in handling heterogeneous traffic tasks, multi-task processing, and pre-training knowledge forgetting. When dealing with heterogeneous traffic, the above methods often struggle to effectively parse and analyze different traffic data simultaneously because they have difficulty in understanding the feature overlaps and conflicts between heterogeneous tasks [8]. This leads to difficulties in accurately capturing the nuances of different traffic patterns. In multi-task processing, LLMs face challenges in balancing and efficiently managing multiple concurrent tasks, which can result in suboptimal performance and potential conflicts between tasks [42]. Furthermore, pre-training knowledge forgetting poses a significant issue, as LLMs can lose previously acquired knowledge during fine-tuning on new tasks [23], diminishing their ability to generalize and apply past learnings to new, related tasks. These limitations highlight the need for specialized approaches and improvements in LLM architectures to enhance their effectiveness in traffic analysis tasks.

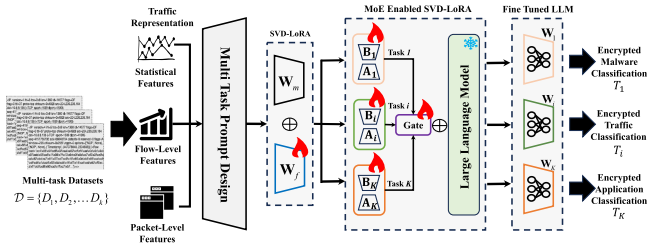


Fig. 2. Workflow of our TrafficLLM service.

IV. OUR APPROACH

In this section, we present TrafficLLM, a general LLM designed to adapt to a range of encrypted traffic tasks, such as malicious traffic detection, attack detection, and application classification, as shown in Fig. 2. TrafficLLM is built around three key components: traffic representation, efficient fine-tuning, and multi-task learning. In traffic representation, we aim to build a general multi-task prompt template to encode heterogeneous input data from traffic analysis tasks into token embeddings for efficient processing by LLMs. For efficient fine-tuning, we design an SVD-LoRA for TrafficLLM to preserve pre-training knowledge and reduce adaptation costs. In multi-task learning, we introduce the MoE network and design a task-aware gating function that assigns different weights to various experts, optimizing multi-task inference performance.

A. Traffic Representation

Data Preprocessing: Traffic data preprocessing involves transforming raw network traffic packets into structured data, represented as five-tuples, to summarize the characteristics of traffic flows [8]. Specifically, let $\mathcal{P}$ represent the pcap file, p_i represent the i-th packet in $\mathcal{P}$, and f_j represent the j-th flow. Each packet p_i in $\mathcal{P}$ is defined by its five-tuple: $p_i = (\text{dst\_IP}_i, \text{src\_IP}_i, \text{dst\_port}_i, \text{src\_port}_i, \text{protocol}_i)$, where dst_IP is the destination IP address, src_IP is the source IP address, dst_port is the destination port number, src_port is the source port number, and protocol indicates the application layer protocol type. We define a flow f_j as a set of packets sharing the same five-tuple: $f_j = \{p_i \in \mathcal{P} \mid (\text{dst\_IP}_i, \text{src\_IP}_i, \text{dst\_port}_i,$

TABLE II
CLASS-SPECIFIC FIELDS RESAMPLING STRATEGY

Field Type	Resampling Method	Class-Specific Considerations
TTL	Kernel Density Estimation (KDE)	BitTorrent typically uses TTL values around 64, while malware like Virut predominantly uses 128
Window Size	Empirical distribution sampling	Streaming applications show larger window sizes than web browsing traffic
Inter-arrival Time	Gaussian mixture modeling	Malware traffic often exhibits periodic patterns unlike benign applications

src_port_i, protocol_i) = (dst_IP_j, src_IP_j, dst_port_j, src_port_j, protocol_j)}. To ensure fairness during training, we anonymize the address information (i.e., MAC address and IP address) in each packet. The result is a set of anonymized data flows $\{f_1, f_2, \ldots, f_m\}$, each represented by packets sharing the same five-tuple, ready for further feature extraction.

Class-Specific Data Augmentation: In practical scenarios, traffic datasets are often unbalanced, with some task labels having significantly fewer data instances compared to others. For instance, in the USTC-TFC [43] dataset used for malicious traffic classification, labels like "Virut" and "Nsis-ay" have substantially fewer instances compared to benign traffic classes. This imbalance can hinder LLMs from accurately learning features, particularly for minority classes. To handle label imbalance while preserving the statistical signatures that define each traffic class, we implement a class-specific augmentation process that generates new traffic samples by resampling header values from realistic, class-specific distributions rather than using global random ranges.

Class-Specific Distribution Modeling: Let Y be the set of all labels in the dataset, and N_Y be the number of data points for label Y. We identify labels $Y_{\text{under}} \subseteq Y$ with significantly fewer data points: $N_{Y_{\text{under}}} \ll \max(N_Y)$. For each underrepresented class Y_{under}, we model the empirical distribution of key header fields from samples belonging to that specific class. Unlike our previous approach that used global randomization ranges, we now extract per-class distributions for fields such as TTL, window size, and inter-arrival times. Specifically, for each class c, we compute:

$$P_c(f) = \frac{\text{count of field } f \text{ values in class } c}{\text{total samples in class } c}, \tag{2}$$

where $P_c(f)$ represents the probability distribution of field f for class c. These distributions are used to guide the resampling process, ensuring augmented samples maintain the statistical characteristics that define each traffic class.

Header Field Modification with Class Preservation: For each packet p_i belonging to an underrepresented label Y_{under}, we extract its IP and TCP headers. Rather than applying random masks with global ranges as in our previous approach, we now resample header values from the class-specific distributions $P_{Y_{\text{under}}}(f)$, as shown in Table II. The modified headers are defined as:

$$H'_{\text{IP}}(p_i) = \begin{cases} f_j \sim P_{Y_{\text{under}}}(f_j), & \text{for } f_j \text{ to be modified;} \\ f_j = H_{\text{IP}}(p_i)[f_j], & \text{otherwise.} \end{cases} \tag{3}$$

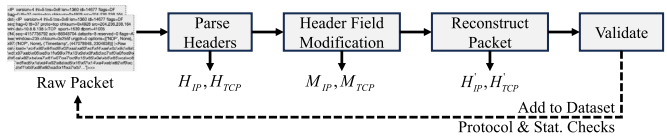


Fig. 3. Overview of data augmentation strategies.

$$H'_{\text{TCP}}(p_i) = \begin{cases} f_j \sim P_{Y_{\text{under}}}(f_j), & \text{for } f_j \text{ to be modified;} \\ f_j = H_{\text{TCP}}(p_i)[f_j], & \text{otherwise,} \end{cases} \tag{4}$$

where $\sim$ denotes sampling from the probability distribution. This approach ensures that the generated samples preserve the distinctive statistical patterns that characterize each traffic class, rather than introducing artificial patterns that models might exploit.

Construction and Validation of Synthetic Samples: After resampling header values, we construct new packets p'_i using the original packet's payload and the modified headers: $p'_i = \text{Packet}(H'_{\text{IP}}(p_i), H'_{\text{TCP}}(p_i), \text{Payload}(p_i))$. Each synthetic sample undergoes rigorous validation: (a) Protocol Conformance Check: Using Scapy to ensure all modified headers comply with IP/TCP standards; (b) Distributional Similarity Validation: We perform Kolmogorov-Smirnov tests between original and augmented samples to confirm statistical similarity (all p-values > 0.05). Fig. 9 demonstrates the distribution similarity for the Virut class before and after augmentation.

Handling Class Imbalance: The augmentation process is repeated iteratively for each underrepresented class Y_{under} until its sample count is balanced with the median class size. Specifically, for each $y \in Y_{\text{under}}$, we generate $N' = \max_{y' \in Y} N_{y'} - N_y$ synthetic packets using the above procedure. The final dataset achieves approximate class balance without sacrificing diversity. The above process can be summarized in Fig. 3.

Feature Construction: In this process, we construct three-dimensional features to improve traffic classification accuracy by capturing comprehensive flow characteristics. For a given flow f_j, the first dimension consists of flow-level information directly extracted from packets, including dst_port_j, src_port_j, protocol_j, and the application-layer protocol, which provide essential classification cues. The second dimension contains statistical packet information, obtained by traversing the flow to compute the total number of packets N_j, total bytes B_j, and flow duration d_j (i.e., the time difference between the last and first packets), offering a global view of flow behavior. The third dimension captures fine-grained information from the first n packets (typically 3–5), including packet length $L_{j,k}$, direction $Dir_{j,k}$, and content $C_{j,k}$, which reflect early traffic behavior patterns. Together, these three dimensions, i.e., flow-level, statistical, and initial packet features, form a robust and holistic representation of each flow, significantly enhancing traffic classification performance.

Multi-task Prompt Template Design: Through the feature construction process described above, we extract key features relevant to encrypted traffic classification. Traditionally, these features are transformed into token-like embeddings by pre-trained models for further processing by LLMs, such as fine-tuning [8], [9], [26]. However, such approaches incur substantial pre-training and adaptation overhead and struggle to efficiently

manage heterogeneous traffic representations. To address this, we adopt instruction fine-tuning (detailed in the next section) and design a multi-task prompt template incorporating task descriptions, enabling the model to better understand and execute diverse tasks. This allows LLMs to focus less on raw traffic features and more on the relationship between task descriptions and features. The Prompt Template (PT) consists of four elements: task description $\mathcal{T}$, input features $\mathcal{F}$, output format $\mathcal{Y}$, and context information $\mathcal{I}$, formally represented as $PT = \{\mathcal{T}, \mathcal{F}, \mathcal{Y}, \mathcal{I}\}$. An example is provided below to illustrate this concept.

Multi-task Prompt Template Example

$\mathcal{T}$ = "The following is an encrypted network traffic data segment, in which the traffic categories are: BitTorrent, Cridex, Facetime, FTP, Geodo, Gmail, Htbot, Miuref, MySQL, Neris, Nsis-ay, Outlook, Shifu, Skype, SMB, Tinba, Virut, Weibo, WorldOfWarcraft, Zeus. Please perform the encrypted traffic classification task:"

$\mathcal{F}$ = "Source port: 443, Destination port: 38295, Protocol: TCP, Bytes: 553, Total time: 0.0, Average packet interval: 276.5, Average packet arrival time: 0.0, Packet rate: 25974.0, Byte rate: 7181818.2, Number of packets: 2. 1st Packet: Packet timestamp: 0.000000, Packet payload length: 483, Packet payload: 1703 ...; 2nd Packet: Packet timestamp: 0.000077, Packet payload length: 70, Packet payload: 4c845a2a."

$\mathcal{Y}$ = "BitTorrent"

$\mathcal{I}$ = "answer_choices: ["BitTorrent", "Cridex", "Facetime", "FTP", "Geodo", "Gmail", "Htbot", "Miuref", "MySQL", "Neris", "Nsis-ay", "Outlook", "Shifu", "Skype", "SMB", "Tinba", "Virut", "Weibo", "WorldOfWarcraft", "Zeus"], task_type: "cls", task_dataset: "USTC-TFC""

Traffic-Domain Tokenization: To bridge the modality gap between natural language and heterogeneous traffic data, `TrafficLLM` introduces a specialized traffic domain tokenizer to process the diverse inputs required for traffic detection and generation tasks, thereby making them compatible with LLMs. This mechanism effectively extends the native token generation capabilities of LLMs by training a dedicated tokenizer on a large-scale traffic domain corpus. To further mitigate the impact of the modality gap and enable LLMs to understand traffic data, `TrafficLLM` incorporates a tailored feature extractor designed to extract task-specific traffic features. These extracted features, as described in the earlier feature construction section, are well-suited for use within the `TrafficLLM` service. After extracting fine-tuning data, we train a domain-specific tokenizer using the Byte Pair Encoding (BPE) [44] algorithm on the large-scale traffic dataset. Since native LLMs are generally unfamiliar with traffic data, this tokenizer effectively serves as an extension to the original tokenizer, enabling the model to better interpret and generate traffic-related content. The following Fig. 4 illustrates the differences between generation results with and without the use of the traffic domain tokenizer.

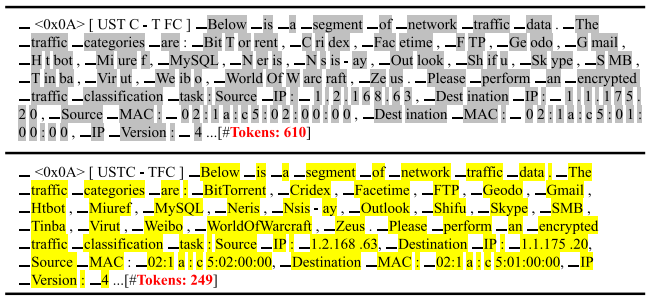


Fig. 4. The difference between traffic domain token generation and native token generation methods.

Discussion: Instruction fine-tuning enhances model performance and mitigates hallucination issues by guiding the model with task-specific instructions [45]. This approach improves adaptability to new tasks, increases efficiency by leveraging pre-trained knowledge, and ensures better alignment with user intent, which can avoid hallucinations in LLMs. Furthermore, it typically requires less additional data, enhances interpretability, and supports flexible deployment across a wide range of applications. By refining the model's behavior for specific tasks, instruction fine-tuning promotes stronger generalization and greater effectiveness in diverse real-world scenarios.

B. SVD-aware LoRA for LLMs

To address C2, LoRA techniques are used to reduce adaptation costs and memory usage. However, they randomly initialize two low-rank matrices and optimize parameters within an unguided subspace [40], which may overwrite important pre-trained features and degrade performance [46]. For example, in encrypted traffic fine-tuning, LoRA-adapted LLMs may lose traffic-related knowledge acquired during pre-training. To address this, we aim to develop a fine-tuning method that better preserves pre-trained features. Driven by the idea of feature matrix decomposition, we aim to design a singular value decomposition (SVD)-aware LoRA (SVD-LoRA) method to efficiently fine-tune LLMs while maximally preserving pre-trained features. We provide the necessary background knowledge on SVD in the first place:

Definition 1 (Singular Value Decomposition): SVD provides a method to decompose a matrix into three other matrices, capturing the essential properties of the original matrix. Thus, the SVD of a real matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ is expressed as:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^{\top}, \tag{5}$$

where $\mathbf{U} \in \mathbb{R}^{m \times m}$ is an orthogonal matrix, whose columns are the left singular vectors, $\mathbf{\Sigma} \in \mathbb{R}^{m \times n}$ is a diagonal matrix with non-negative real numbers on the diagonal (singular values) and zeros elsewhere, and $\mathbf{V} \in \mathbb{R}^{n \times n}$ is an orthogonal matrix, whose columns are the right singular vectors. The singular values σ_i on the diagonal of $\mathbf{\Sigma}$ follow the ordering: $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_{\min(m,n)} \geq 0$.

According to the above definition, given a parameter matrix $\boldsymbol{W} \in \mathbb{R}^{m \times n}$ of a pre-trained model f_w, it can be decomposed in SVD form as follows: $\boldsymbol{W} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^{\top}$. Let $\boldsymbol{U} = [u_1, u_2, \ldots, u_m]$

and $\boldsymbol{V} = [v_1, v_2, \ldots, v_n]$, thus, the SVD of $\boldsymbol{W}$ can be reformulated as follows:

$$\boldsymbol{W} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\top = \sum_{i=1}^{m} \sigma_i u_i v_i^\top, \tag{6}$$

where u_i and v_i are the i^{th} column of $\boldsymbol{U}$ and $\boldsymbol{V}$, respectively.

SVD-LoRA: To effectively preserve the main features of the pre-trained model while simultaneously acquiring new features from the fine-tuning dataset, our key insight suggests decomposing the linear weight matrix, denoted as $\boldsymbol{W}$ of the pre-trained model, into two distinct components: a Main Task Matrix $\boldsymbol{W}_m$ and a Fine-Tuning Matrix $\boldsymbol{W}_f$. The former component is tasked with maintaining the pre-training features, whereas the latter assumes responsibility for learning the unique features associated with the new task at hand. This conceptual decomposition can be mathematically articulated as follows:

$$\boldsymbol{W} = \boldsymbol{W}_m + \boldsymbol{W}_f = \sum_{i=1}^{m-r} \sigma_i u_i v_i^\top + \sum_{i=m-r+1}^{m} \sigma_i u_i v_i^\top, \tag{7}$$

where r is the hyperparameter, i.e., the number of minor singular values in the $\boldsymbol{W}_m$ matrix. Let $\boldsymbol{U}_m = [u_1, u_2, \ldots, u_{m-r}]$, $\boldsymbol{U}_f = [u_{m-r+1}, u_{m-r+2}, \ldots, u_m]$, $\boldsymbol{V}_m = [v_1, v_2, \ldots, v_{m-r}]$, and $\boldsymbol{V}_f = [v_{m-r+1}, v_{m-r+2}, \ldots, v_m]$, according to (5), we rewrite (7) as follows:

$$\boldsymbol{W} = \boldsymbol{U}_m\boldsymbol{\Sigma}_m\boldsymbol{V}_m^\top + \boldsymbol{U}_f\boldsymbol{\Sigma}_f\boldsymbol{V}_f^\top = \boldsymbol{W}_m + \boldsymbol{W}_f. \tag{8}$$

We then rewrite $\boldsymbol{W}_f$ following the idea of the LoRA method:

$$\boldsymbol{W}_f = \left(\boldsymbol{U}_f\sqrt{\boldsymbol{\Sigma}_f}\right)\left(\sqrt{\boldsymbol{\Sigma}_f}\boldsymbol{V}_f^\top\right) = \boldsymbol{B}_f\boldsymbol{A}_f. \tag{9}$$

Thus, we have: $\boldsymbol{W} = \boldsymbol{W}_m + \boldsymbol{B}_f\boldsymbol{A}_f$. Therefore, we can freeze the main task matrix $\boldsymbol{W}_m$ to retain important pre-trained features and learn a trainable low-rank fine-tuning matrix $\boldsymbol{B}_f\boldsymbol{A}_f$ to learn the features of the new task, thereby achieving efficient LLMs fine-tuning.

Discussion: Unlike vanilla LoRA, SVD-LoRA constructs a learnable subspace to capture new task knowledge instead of randomly initializing low-rank matrices. This reduces interference between fine-tuning updates and pre-trained knowledge. The advantages of SVD-LoRA are twofold: 1) *Subspace Fine-Tuning*: it retains pre-trained knowledge while efficiently learning new information, balancing stability and adaptability; 2) *Parameter Efficiency*: trainable parameters are sparse and comparable in number to LoRA, enabling efficient fine-tuning and compatibility with existing hardware (e.g., GPUs).

C. MoE Enabled SVD-LoRA for Multi-Task Learning

MoE Enabled SVD-LoRA: This section presents a MoE-enabled SVD-LoRA robust multi-task learning module for `TrafficLLM`, designed to reduce interference between encrypted traffic classification tasks and enable efficient multi-task learning. Unlike LoRA, which fine-tunes all parameters for each task, MoE-enabled SVD-LoRA partitions the parameter set and employs a MoE network to select task-specific combinations. This approach captures shared knowledge while integrating different experts for specific tasks. Let the group of experts be denoted as $\{E_i\}_{i=1}^{N}$, responsible for learning the fine-tuning matrix $\Delta\mathbf{W}_f$. Each expert in the SVD-LoRA layer is implemented as two decomposed low-rank matrices. For samples from task $\mathcal{T}_j$, the forward pass of the linear layer with the SVD-LoRA layer is expressed as:

$$\begin{aligned}\mathbf{h}_j &= \mathbf{W}_m\mathbf{x}_j + \Delta\mathbf{W}_{f_j}\mathbf{x}_j \\ &= \mathbf{W}_m\mathbf{x}_j + \sum_{i=1}^{N}\omega_{ji}\cdot E_i(\mathbf{x}_j) \\ &= \mathbf{W}_m\mathbf{x}_j + \sum_{i=1}^{N}\omega_{ji}\cdot \mathbf{B}_i\mathbf{A}_i\mathbf{x}_j\end{aligned} \tag{10}$$

where $\mathbf{h}_j$ and $\mathbf{x}_j$ denote the input and output of intermediate LLM layers for samples from task T_j. Each expert E_i is comprised of matrices $\mathbf{B}_i \in \mathbb{R}^{m\times\frac{r}{N}}$ and $\mathbf{A}_i \in \mathbb{R}^{\frac{r}{N}\times n}$. Note that $\mathbf{B}_i$ and $\mathbf{A}_i$ are split by the low-rank matrices $\mathbf{B}_f$ and $\mathbf{B}_f$ according to the number of experts. The hyper-parameter N represents the number of experts in MoE-enabled SVD-LoRA, with each expert's matrices $\mathbf{A}$ and $\mathbf{B}$ having a rank of $\frac{r}{N}$. To ensure that distinct parameters are learned for different tasks, we need to dynamically adjust the contribution weights ω_{ji} of different experts for a specific task T_j, where the weights are determined by our proposed gating function module (see below for details).

Task-aware Gate Function Design: To adapt the MoE-enabled SVD-LoRA to specific tasks, it is necessary to assign task-specific contribution weights to the experts. We introduce two task-aware gating functions: a dense gating function, which integrates contributions from all experts, and a sparse gating function, which selectively integrates the top κ experts with the largest contributions [47], [48], [49]. To implement these functions, we use the task representation and a transformation matrix as input. By pairing the task embedding with the transformation matrix, a linear layer maps this combination to the experts' contribution weights. Specifically, let $\mathbf{T} \in \mathbb{R}^{|\mathcal{T}|\times d_T}$ denote the task embedding matrix (d_T is the embedding dimension), and $\mathbf{W}_T \in \mathbb{R}^{N\times d_T}$ denote the transformation matrix. Based on this setup, we define the two gating functions as follows.

For dense gating functions, we extract the j-th column of the task representation matrix $\mathbf{T}$ as the representation vector of the task, symbolized by $\mathbf{t}_j \in \mathbb{R}^{d_T}$. Next, we combine the transformation matrix $\mathbf{W}_T$ with the task representation $\mathbf{t}_j$ to form $\mathbf{z_j} = \mathbf{W}_T\mathbf{t}_j$. To determine the contribution weights for task T_j, we apply a linear transformation. This computation is captured by the following equations:

$$\boldsymbol{\omega}_j = \text{Softmax}(\mathbf{z}) = \frac{\exp(\mathbf{z}_j)}{\sum_{j=1}^{N}\exp(\mathbf{z}_j)}, \tag{11}$$

where the softmax operation is aim to obtain the normalized contribution weights.

Similar to the dense approach, the contribution of each expert is tailored to specific tasks, but only a subset of experts contributes to the final output. We use the same task embedding matrix $\mathbf{T} \in \mathbb{R}^{|\mathcal{T}|\times d_T}$ and follow a similar process to extract the task representation $\mathbf{t}_j \in \mathbb{R}^{d_T}$ for task T_j. Next, we select the top κ experts based on the computed gating weights. The sparse gating function can be formally defined as follows:

$$\boldsymbol{\omega}_j = \text{Softmax}(\text{Top}(\mathbf{z}_j, \kappa)), \tag{12}$$

$$\text{Top}(\mathbf{x}, \kappa) = \begin{cases} x_i & \text{if } x_i \text{ in top } \kappa \text{ elements of } \mathbf{x}. \\ 0 & \text{otherwise.} \end{cases} \tag{13}$$

Algorithm 1: Pseudocode of the `TrafficLLM`

Data Preparation
1: Perform data preprocessing and partition using five-tuples.
2: Perform data augmentation via random masking.
3: Extracting traffic representations from network flow data.
Fine-tuning Process
4: Perform SVD to obtain $\mathbf{W}_m$ and $\mathbf{W}_f$.
5: Freeze the matrix $\mathbf{W}_m$ to retain knowledge.
6: Split the fine-tuning matrix $\mathbf{W}_f$ via # of experts.
7: **for** a batch of samples B in $\mathcal{D}$ **do**
8: Perform forward process for `TrafficLLM` accompanied with MoE enabled SVD-LoRA via (10).
9: Perform the optimization steps via calculation (1).
10: Perform gradient descent to optimize the MoE parameter matrices $\mathbf{A}_i$ and $\mathbf{B}_i$ and the gating-related task representation matrices $\mathbf{E}$ and $\mathbf{W}_T$.
11: **end for**
Inference Process
10: **for** T_j in $\mathcal{T}$ **do**
11: Perform expert contribution weight calculation via (11)–(13).
12: Perform fine-tuned parameter calculation via (14) for each task.
13: **end for**
14: Perform multi-task inference via identifying task identifiers.

While the traditional design of MoE directly feeds the input vector $\mathbf{x}$ into the gate function, our approach is the same as the current mainstream approach [47], [48], [49], that is, we only input the task identity into the gate function. Subsequently, in order not to introduce additional overhead, we aim to generate a unique set of model parameters for each task through linear transformation. Finally, when task T_j performs inference, we combine the knowledge of different experts and use fine-tuned parameters. The process can be formally defined as follows:

$$\mathbf{W}_j = \mathbf{W}_m + \Delta\mathbf{W}_f = \mathbf{W}_m + \sum_{i=1}^{N} \omega_{ji} \cdot \mathbf{B}_i\mathbf{A}_i. \tag{14}$$

Discussion: We highlight the benefits of the task-aware gating design. Compared to input representation-aware gating, our method does not require configuring expert weights for each input sample or introducing additional parameters for nonlinear transformations. Expert weights are computed solely from the task representation matrix and used with (14) to obtain fine-tuned parameters for reasoning. Consequently, task-aware gating achieves task-specific configuration without extra overhead, avoiding high adaptation costs and latency. Implementation details of `TrafficLLM` are provided in Algorithm 1.

D. Cost Analysis

We analyze the cost of the proposed MoE-enabled SVD-LoRA in terms of trainable parameters and model complexity, and compare it with classic LoRA.

Number of Trainable Parameters: LoRA introduces two low-rank matrices $\mathbf{B} \in \mathbb{R}^{m\times r}$ and $\mathbf{A} \in \mathbb{R}^{r\times n}$, where $r \ll \min(m, n)$, yielding $\text{Parameters}_{\text{LoRA}} = r \cdot (m + n)$. In MoE-enabled SVD-LoRA, each of the N experts E_i contains $\mathbf{B}_i \in \mathbb{R}^{m\times \frac{r}{N}}$ and $\mathbf{A}_i \in \mathbb{R}^{\frac{r}{N}\times n}$, giving $\text{Parameters}_{\text{MoE-SVD-LoRA}} = N \cdot (m \cdot \frac{r}{N} + \frac{r}{N} \cdot n) = r \cdot (m + n)$. Thus, MoE-enabled SVD-LoRA has the same number of trainable parameters as LoRA while supporting superior multi-task performance.

Model Complexity: For LoRA, the complexity of updating the low-rank matrices is $\text{Complexity}_{\text{LoRA}} = O(m \cdot r \cdot n)$. Similarly, the forward and backward passes through all experts in MoE-SVD-LoRA yield the same complexity. SVD decomposition adds $\text{Complexity}_{\text{SVD}} = O(\min(mn^2, m^2n))$. Combining these, the total complexity is

$$\text{Complexity}_{\text{MoE-SVD-LoRA}} = O(m \cdot r \cdot n + \min(mn^2, m^2n)). \tag{15}$$

Since the gating function is integrated into the linear layer, it introduces no additional parameters or forward/backward cost. Although SVD increases complexity compared to LoRA, MoE-SVD-LoRA preserves pre-trained knowledge and resolves conflicts in multi-task learning, offering enhanced flexibility and efficiency. Therefore, a trade-off exists between multi-task performance, knowledge preservation, and model complexity to enable LLMs to adapt effectively to diverse applications.

V. Experiments

A. Experiment Setup

To evaluate the performance of our `TrafficLLM` system, we conduct extensive experiments on 7 benchmarking datasets. All experiments are developed using Python 3.9 and PyTorch 1.12 and evaluated on a server with an NVIDIA A100 GPU.

Datasets and Downstream Tasks: To comprehensively evaluate the performance and learning capabilities of `TrafficLLM`, we consider the following datasets and downstream tasks:

Task 1: General Encrypted Application Classification: This task targets classifying application traffic under standard encryption protocols. We use Cross-Platform (iOS) [50] and Cross-Platform (Android) [50] datasets, with 196 and 215 applications respectively, sourced from the top 100 apps in the US, China, and India. *Task 2: Encrypted Malware Classification:* This task aims to differentiate between malware and benign applications in encrypted traffic. We utilize the USTC-TFC dataset [43], which includes 10 categories of benign and malicious traffic.

Task 3: Encrypted Traffic Classification on VPN: This task focuses on classifying encrypted traffic that utilizes VPNs. We employ the ISCX-VPN dataset [51], comprising traffic from 6 communication applications captured by the Canadian Institute for Cybersecurity in both VPN and non-VPN modes. For evaluating `TrafficLLM`, we further divide this dataset into ISCX-VPN-Service with 12 service categories and ISCX-VPN-App with 17 application categories.

Task 4: Encrypted Application Classification on Tor: This task aims to classify encrypted traffic using The Onion Router (Tor) for enhanced privacy. The ISCX-Tor dataset [52], which includes 16 applications, is used for this task. *Task 5: Encrypted Application Classification on TLS 1.3:* This task focuses on classifying encrypted traffic using the TLS 1.3 protocol. The dataset, CSTNET-TLS 1.3 [9], comprises 120 applications collected by CSTNET from March to July 2021, making it the first known TLS 1.3 dataset.

To ensure fair comparison and realistic evaluation of multi-task learning capabilities, we implement a strict data partitioning protocol that prevents leakage between training and testing environments. To this end, we partition datasets such that training and testing sets contain traffic from entirely different devices, capture sessions, and time windows, specifically, for VPN and Tor datasets we use non-overlapping time periods, while for Cross-Platform and malware datasets we ensure no device or capture context appears in both sets. We adopt a 7:3 ratio of training and testing for all datasets, following previous work [8], [11], [36]. This protocol better reflects real-world deployment scenarios where models must generalize to unseen network environments.

Baselines: In this paper, we compare the following baselines:

FS-Net [35]: FS-Net combines feature selection and deep learning for classifying encrypted network traffic. It focuses on extracting relevant features to enhance accuracy and robustness against encryption.

AppScanner [3]: AppScanner uses machine learning to identify mobile apps based on their network traffic patterns, even when the traffic is encrypted, aiding in monitoring and managing app usage.

FlowPic [12]: FlowPic converts network flows into images, leveraging convolutional neural networks to classify encrypted traffic by extracting spatial patterns from the visual data.

PERT [33]: PERT focuses on packet-level analysis of encrypted traffic using deep learning, capturing fine-grained patterns for improved classification accuracy where traditional methods may fail.

TFE-GNN [36]: TFE-GNN uses graph neural networks to represent and classify network traffic as graphs, effectively capturing dependencies and patterns in encrypted traffic.

ET-BERT [9]: ET-BERT adapts the BERT model for encrypted traffic classification, leveraging transformers to understand long-range dependencies and improve performance in encrypted environments.

NetGPT [8]: NetGPT applies the GPT model to network traffic classification, using transformer architecture to analyze patterns and predict future traffic, handling both unencrypted and encrypted data.

DeepSeek [53]: DeepSeek is one of the most advanced LLMs, which has achieved excellent performance on various tasks. We use LoRA method to fine-tune it.

ChatGLM2-6B [37]: ChatGLM2-6B is an LLM with 6 billion parameters, designed for natural language tasks but adaptable for analyzing sequential data in network traffic analysis. Note that we use ChatGLM2-6B with full parameter fine-tuning (denoted as ChatGLM (F)) and LoRA fine-tuning (denoted as ChatGLM (L)) as our baselines. In addition, we only fine-tune datasets related to a single traffic task.

For all LLMs, including ET-BERT, NetGPT, ChatGLM variants, and DeepSeek, we standardize the input format using identical multi-task prompt templates that convert raw traffic features into natural language instructions. Each task is identified by a specific prefix in the prompt (e.g., "Task: Classify VPN Service; Input: {flow features}"), ensuring all models receive semantically equivalent inputs regardless of their architecture.

Hyperparameters: Considering practical network deployment, we compare the parameter size, memory usage, and traffic classification performance of different LLMs, summarized in Table III. Based on these results, we select the open-source ChatGLM2-6B [37] as our base LLM. Due to computational and resource constraints, we do not explore the impact of the proposed fine-tuning method on other base LLMs in this paper. For LoRA fine-tuning, key hyperparameters are set as follows: learning rate 0.0001, maximum input tokens 1024, maximum output tokens 120, batch size 64, and 5000 training steps. For SVD, the number of singular values r is set to 16, and for MoE, the number of dense experts is 8 and sparse experts 2, unless otherwise specified. All hyperparameters are determined through repeated fine-tuning and comparison.

Evalution Metrics: We assess and compare the performance of NetMamba using four typical metrics: Accuracy (AC), Precision (PR), Recall (RC), and weighted F1 Score (F1).

B. Numerical Results

Next, we conduct experiments to evaluate the effectiveness of the designed `TrafficLLM` service above in various situations. Our evaluation primarily aims to answer the following Research Questions (RQ):

- [RQ1] How does the overall performance of `TrafficLLM` compare to other large models and different fine-tuning strategies?
- [RQ2] How does `TrafficLLM` compare to other large models in terms of the number of trainable parameters, throughput, memory usage, and latency?
- [RQ3] How do key hyperparameters affect the performance of `TrafficLLM`?
- [RQ4] How does `TrafficLLM` perform on unseen traffic datasets and general knowledge datasets compared with other large models?
- [RQ5] How do the components in `TrafficLLM` affect its performance?
- [RQ6] How does `TrafficLLM` perform when actually deployed in an open environment?

Overall Performance (RQ1): In this experiment, we conducted comparative experiments across 7 datasets and 5 traffic analysis tasks. Tables V and VI summarize the results, revealing that `TrafficLLM` consistently outperforms competing methods in both single and multi-task settings. On Cross-Platform datasets, `TrafficLLM (D)` achieves 0.9657 and 0.9701 accuracy on iOS and Android respectively, surpassing the strongest baseline (ET-BERT) by 1.32 and 0.48 percentage points. For VPN traffic analysis, our method attains 0.9741 accuracy on

TABLE III
COMPARISON OF LLMS IN TERMS OF PARAMETER SIZE, MEMORY USAGE, AND PERFORMANCE ON TRAFFIC CLASSIFICATION TASKS

Model	Parameters (Approx.)	FP16 Inference VRAM	Quantized VRAM (INT8/INT4)	Encrypted Traffic Performance
ChatGLM-6B	6B	12GB	6GB (INT8), 3GB (INT4)	Medium (70%–80%)
GPT-4	1.8T (MoE)	High (multi-GPU)	Depends on tools (e.g., TensorRT)	High (85%–90%)
DeepSeek 1.0	1.2T	High (multi-GPU)	Supports compression	High (80%–88%)
Llama / Llama2 / Llama3	7B / 13B / 70B	14GB / 26GB / 140GB	7GB / 13GB / 35GB (INT4)	Medium-High (75%–85%)
Llama3-Chinese-8B	8B	15GB	4GB (INT4)	Medium-High (75%–83%)
Qwen Series	7B / 14B / 72B	14GB / 28GB / 140GB	Supports INT4	Medium-High (75%–85%)
Baichuan Series	7B / 13B	14GB / 26GB	Supports INT4	Medium (70%–80%)
Yi Series	6B / 9B / 34B	12GB / 18GB / 70GB	Supports INT4	Low (65%–75%)

TABLE IV
THE STATISTICAL INFORMATION OF THE DATASETS

Task	Dataset	# of Flow	# of Packet	# of Label
Task 1	Cross-Platform(iOS) [50]	20,858	707,717	196
	Cross-Platform(Android) [50]	27,846	656,044	215
Task 2	USTC-TFC [43]	9,853	97,115	20
Task 3	ISCX-VPN-Service [51]	3,694	60,000	12
	ISCX-VPN-App [51]	2,329	77,163	17
Task 4	ISCX-Tor [52]	3,021	80,000	16
Task 5	CSTNET-TLS 1.3 [9]	46,372	581,709	120

service classification and 0.8735 on application identification, exceeding previous state-of-the-art by 1.68 and 1.47 percentage points. ChatGLM variants consistently underperform across all tasks, particularly on ISCX-Tor where `TrafficLLM (D)` achieves 0.9720 accuracy compared to ChatGLM (L)'s 0.9030. While ET-BERT shows competitive results on malware classification (USTC-TFC), it requires substantially more computational resources. The dense and sparse gating variants demonstrate complementary strengths: dense gating excels in most multi-task scenarios by leveraging all experts' knowledge, while sparse gating achieves superior performance on specific challenging tasks like USTC-TFC (0.9780 F1-score). These results demonstrate that `TrafficLLM`'s architecture effectively balances adaptation capability with computational efficiency across diverse encrypted traffic analysis tasks. Additional confusion matrix results can be found in the Appendix B, available online.

Efficiency Performance Analysis (RQ2): We compared the trainable parameters, throughput, memory usage, and latency of ET-BERT and NetGPT with `TrafficLLM`. Considering that ChatGLM served as the base LLM for this paper and traditional DL methods did not involve prompt processing, we excluded them from efficiency performance comparisons. Specifically, to ensure fair comparison of throughput and latency, we used the maximum number of training samples processed per unit time (i.e., 1 s) to quantify throughput and measured the time required to process and respond to a single sample request to quantify inference latency. Additionally, since ET-BERT functioned as a single-task model, we accumulated its memory usage across tasks for fair comparison. Regarding trainable parameters, we compared ChatGLM fine-tuned using LoRA, SVD-LoRA, and MoE-SVD-LoRA against baselines to demonstrate how different fine-tuning methods affected parameter efficiency. The experimental results reported in Fig. 5 show that `TrafficLLM` maintains better inference efficiency than ET-BERT and NetGPT, though its memory usage and throughput performance remain inferior to these baselines. For instance, the throughput of `TrafficLLM` is approximately $1/3$ of ET-BERT's throughput, yet its inference latency remains lower than ET-BERT's. Overall, `TrafficLLM` achieves an effective balance across these efficiency metrics, demonstrating practical applicability for real-world deployment scenarios.

Model Performance under Different Hyperparameters (RQ3): We explored the impact of two key parameters, i.e., the number of experts $N \in \{2, 4, 8, 10\}$ and the number of singular values $r \in \{4, 8, 16, 32\}$, on the performance of `TrafficLLM`. Specifically, we fixed r to study the effect of N, and vice versa when examining r. Fig. 6 summarizes the experimental results and indicates two conclusions: 1) Model performance does not always increase with the number of experts. The optimal number of experts is $N = 8$, as too many experts reduce the LoRA rank for each expert, diminishing the learning ability of the low-rank matrix. 2) Model performance gradually improves with increasing r, but the number of trainable parameters also rises significantly. Considering the performance-efficiency trade-off, $r = 16$ is the most economical choice.

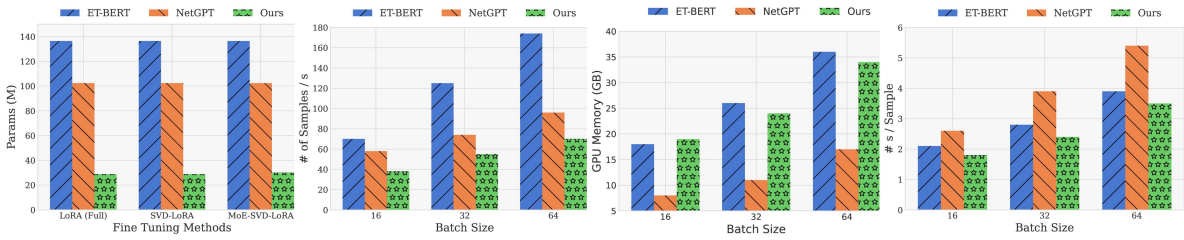
Generalization Performance (RQ4): We conducted sequential training experiments where tasks were learned incrementally ((T_1) Cross-Platform (iOS) $\rightarrow$ (T_2) Cross-Platform (Android) $\rightarrow$ (T_3) ISCX-VPN-Service $\rightarrow$ (T_4) ISCX-VPN-App $\rightarrow$ (T_5) USTC-TFC $\rightarrow$ (T_6) ISCX-Tor $\rightarrow$ (T_7) CSTNET-TLS 1.3), measuring performance degradation on previously learned tasks. We measured accuracy degradation on each previous task and compute average forgetting as: *Forgetting Rate = (Accuracy before new task - Accuracy after new task) / Accuracy before new task*. We created a diagnostic quiz with 100 traffic-domain questions (see Appendix C, available online) focusing on fundamental network concepts, protocol behaviors, and security patterns. This quiz tests retained knowledge that isn't directly tied to any single classification task but is essential for comprehensive traffic understanding. Fig. 7 shows the forgetting curves for the three variants. `TrafficLLM (D)` demonstrates remarkably stable performance with minimal forgetting (average 3.2% degradation), while ChatGLM (F) experiences severe

TABLE V
COMPARISON RESULTS ON CROSS-PLATFORM, ISCX-VPN-SERVICE AND ISCX-VPN-APP DATASETS. WE HIGHLIGHT THE BEST RESULTS AND UNDERLINE THE SUBOPTIMAL RESULTS (THIS ALSO APPLIES TO THE TABLE BELOW).

Dataset	Cross-Platform (iOS)				Cross-Platform (Android)				ISCX-VPN-Service				ISCX-VPN-App			
Method	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1
FS-Net [35]	0.3487	0.2505	0.2458	0.2335	0.4665	0.3363	0.3203	0.2970	0.6935	0.7210	0.7084	0.6737	0.6331	0.4627	0.4662	0.4500
AppScanner [3]	0.2994	0.1848	0.1937	0.1772	0.3596	0.2345	0.2386	0.2167	0.6941	0.7032	0.7035	0.6894	0.5998	0.4705	0.4926	0.4701
FlowPic [12]	0.9091	0.9098	0.8911	0.8898	0.8487	0.8837	0.8411	0.8414	0.7788	0.7793	0.7655	0.7438	0.8565	0.6414	0.6439	0.6227
PERT [33]	0.9530	0.9434	0.9267	0.9229	0.9434	0.8299	0.8321	0.8166	0.9184	0.9211	0.9190	0.9103	0.8001	0.6888	0.6857	0.6721
TFE-GNN [36]	0.8035	0.8067	0.8063	0.7770	0.7976	0.7961	0.7837	0.7827	0.7054	0.7189	0.6947	0.6785	0.6343	0.4654	0.4626	0.4514
ET-BERT [9]	0.9525	0.9426	0.9416	0.9430	0.9653	0.9109	0.8970	0.8918	0.9402	0.9512	0.9557	0.9390	0.8217	0.7246	0.6990	0.7007
NetGPT [8]	0.9492	0.9478	0.9534	0.9456	0.9633	0.9501	0.9576	0.9463	0.9435	0.9504	0.9512	0.9386	0.8233	0.7527	0.7565	0.7560
ChatGLM [37] (F)	0.9511	0.9399	0.9351	0.9273	0.9557	0.9160	0.8957	0.8946	0.9432	0.9438	0.9528	0.9526	0.8588	0.8273	0.7980	0.7934
ChatGLM [37] (L)	0.9585	0.9365	0.9354	0.9440	0.9538	0.9410	0.9352	0.9450	0.9398	0.9369	0.9326	0.8974	0.8256	0.8165	0.7898	0.7797
DeepSeek [53] (L)	0.9471	0.9472	0.9335	0.9372	0.9567	0.9481	0.9343	0.9517	0.9573	0.9407	0.9304	0.9153	0.8403	0.8263	0.8039	0.7801
Ours (D)	**0.9657**	0.9609	**0.9611**	0.9574	**0.9701**	0.9555	**0.9669**	**0.9637**	**0.9741**	**0.9619**	**0.9605**	**0.9595**	**0.8735**	**0.8769**	0.8715	**0.8681**
Ours (S)	0.9503	**0.9650**	0.9552	**0.9646**	0.9654	**0.9692**	0.9540	0.9494	0.9669	0.9596	0.9571	0.9473	0.8735	0.8749	**0.8805**	0.8577

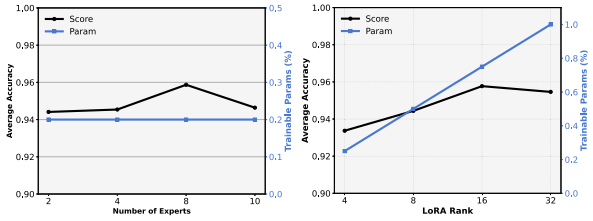
TABLE VI
COMPARISON RESULTS ON ISCX-TOR, USTC-TFC AND CSTNET-TLS 1.3 DATASETS

Dataset	ISCX-Tor				USTC-TFC				CSTNET-TLS 1.3			
Method	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1
FS-Net [35]	0.5821	0.4780	0.5010	0.4290	0.8624	0.8620	0.8690	0.8590	0.8412	0.8180	0.8120	0.8080
AppScanner [3]	0.6480	0.3520	0.4150	0.3680	0.8732	0.8750	0.8740	0.8650	0.6420	0.6010	0.6090	0.5960
FlowPic [12]	0.8850	0.3580	0.3420	0.3410	0.7920	0.6210	0.6780	0.6350	0.1020	0.1120	0.1040	0.0920
PERT [33]	0.7420	0.4180	0.4200	0.4100	0.9720	0.9720	0.9720	0.9710	0.8680	0.8610	0.8490	0.8510
TFE-GNN [36]	0.9650	0.9520	**0.9700**	0.9600	0.9540	0.9540	0.9540	0.9520	0.9620	**0.9620**	0.9620	0.9610
ET-BERT [9]	0.9320	0.9020	0.9380	0.9170	0.9750	0.9750	0.9750	0.9740	0.9310	0.9260	0.9220	0.9230
NetGPT [8]	0.9280	0.9100	0.9220	0.9180	0.9730	0.9730	0.9730	0.9720	0.9020	0.9080	0.9120	0.9160
ChatGLM [37] (F)	0.9290	0.9520	0.9280	0.9290	0.9740	0.9750	0.9760	0.9770	0.9040	0.8990	0.8760	0.8930
ChatGLM [37] (L)	0.9030	0.9340	0.9270	0.9170	0.9580	0.9470	0.9490	0.9300	0.9050	0.9020	0.8740	0.8900
DeepSeek [53] (L)	0.9520	0.9450	0.9440	0.9470	0.9480	0.9530	0.9510	0.9600	0.9580	0.9460	0.9460	0.9400
Ours (D)	**0.9720**	**0.9710**	0.9690	**0.9700**	**0.9810**	0.9790	**0.9770**	0.9760	**0.9670**	0.9580	0.9640	0.9610
Ours (S)	0.9700	0.9670	0.9680	0.9660	0.9760	**0.9790**	0.9750	**0.9780**	0.9620	0.9570	**0.9650**	**0.9630**



(a) # of Trainable Parameters (b) Throughput (c) Memory Usage (d) Inference Latency

Fig. 5. Comparison of experimental results of TrafficLLM with other large models in terms of the number of trainable parameters, throughput, memory usage, and latency.



(a) Impact of Number of Experts (b) Impact of Low Rank

Fig. 6. The results of experiments for hyper-parameters, i.e., # of experts N and LoRA rank r.

forgetting (average 34.7% degradation) after learning all seven tasks. ChatGLM (L) shows intermediate performance (average 19.8% degradation), confirming that parameter-efficient methods help mitigate forgetting, but our SVD-LoRA integration provides substantially better knowledge preservation.

Furthermore, we evaluated models on both general knowledge (ProtoQA [54]) (tested with 100 pieces of data) and unseen traffic data (CICIoT2022 [55]) after sequential adaptation. The questions in the ProtoQA dataset are usually common sense questions such as "Where does the sun rise?". Fig. 8 records the above results and shows that TrafficLLM can not only preserve the pre-training dataset well but also outperform other baselines on unseen tasks, thanks to the SVD processing of the pre-trained model and the multi-task MoE design. For example, the accuracy on the ProtoQA dataset is 62% higher than that of ChatGLM (F).

TABLE VII
THE EXPERIMENTAL RESULTS OF THE ABLATION STUDY FOR TrafficLLM

Model	Cross-Platform (iOS)	Cross-Platform (Android)	ISCX-VPN-Service	ISCX-VPN-App	ISCX-Tor	USTC-TFC	CSTNET-TLS 1.3	Average AC
w/o DA	0.9425	0.9517	0.9523	0.8341	0.9528	0.9685	0.9532	0.9364
w/o MOE	0.9487	0.9546	0.9587	0.8417	0.9582	0.9731	0.9575	0.9418
w/o Gate	0.9564	0.9638	0.9654	0.8523	0.9642	0.9754	0.9619	0.9485
w/o SVD	0.9591	0.9672	0.9701	0.8612	0.9687	0.9789	0.9654	0.9529
w LoRA	0.9512	0.9537	0.9612	0.8467	0.9432	0.9713	0.9512	0.9398
Ours (D)	**0.9657**	**0.9701**	**0.9741**	**0.8735**	**0.9720**	**0.9810**	**0.9670**	**0.9576**

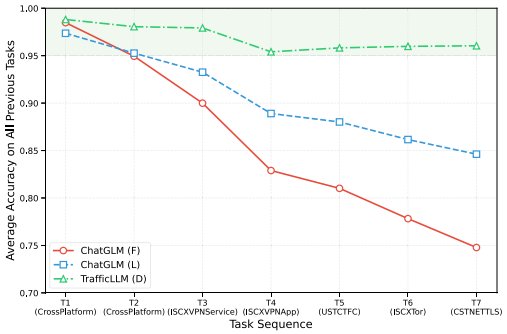
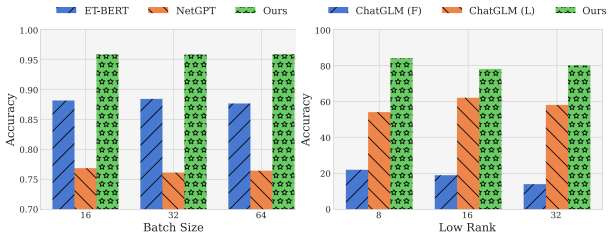


Fig. 7. Forgetting curves during sequential task learning.



(a) Performance on Unseen Dataset (b) Performance on ProtoQA

Fig. 8. Comparative experimental results on unseen tasks and general knowledge tasks.

Ablation Studies (RQ5): We conducted ablation experiments to evaluate the effectiveness of each component. Specifically, we examined the impact of the MoE, gating function, and SVD module on the performance of TrafficLLM. We evaluated the model on seven benchmark datasets under five settings: *w/o* Data Augmentation (DA), *w/o* MoE, *w/o* gating, *w/o* SVD, and *w* LoRA. Table VII shows that the MoE module and gating function effectively improve TrafficLLM's performance, as observed by comparing *w* LoRA with *w/o* gating and *w/o* SVD. The SVD module shows no clear advantage in performance improvement, yet it remains an integral part of TrafficLLM. Additionally, the data augmentation strategy consistently enhances model performance, as omitting it leads to a noticeable drop (see Appendix B, available online).

Furthermore, to validate the performance gains of the designed traffic tokenizer, we tested TrafficLLM and other LLMs on seven benchmark datasets. Specifically, we first evaluated all models using the local ChatGLM tokenizer with the same PT, without a domain-specific vocabulary. Then, we assessed whether the tokenizer benefited all models by replacing it with our traffic domain tokenizer while keeping the same prompt template. The results (see Table VIII) show that the traffic domain tokenizer consistently improves the performance of all models, with accuracy gains ranging from +0.89% to +1.19%. Even without a specialized tokenizer (using only the shared PT), TrafficLLM (D) achieved an average accuracy of 97.15%, which is 1.29% higher than the best baseline (DeepSeek-L at 95.86%). This confirms that our architectural innovations (SVD-LoRA + MoE) are the main contributors to the performance improvement.

TABLE VIII
ABLATION STUDY ON THE IMPACT OF TRAFFIC-DOMAIN TOKENIZER ACROSS DIFFERENT MODELS. RESULTS SHOW AVERAGE ACCURACY (%) ACROSS ALL SEVEN BENCHMARK DATASETS

Method	w/ PT		w/ PT + Tokenizer		Δ Accuracy	
	AC	F1	AC	F1	AC	F1
ET-BERT [9]	94.72	93.85	95.61	94.82	+0.89	+0.97
NetGPT [8]	95.34	94.56	96.53	95.78	+1.19	+1.22
ChatGLM [37] (F)	93.87	92.94	94.86	94.01	+0.99	+1.07
ChatGLM [37] (L)	94.15	93.28	95.24	94.46	+1.09	+1.18
DeepSeek [53] (L)	95.86	94.97	96.92	96.15	+1.06	+1.18
Ours (D)	**97.15**	**96.43**	**98.26**	**97.51**	+1.11	+1.08
Ours (S)	96.87	96.12	97.95	97.24	+1.08	+1.12

TABLE IX
REAL-WORLD DEPLOYMENT PERFORMANCE RESULTS ON THE NUDT-MOBILE DATASET

Model	AC	PR	RC	F1	Throughput	Latency
Ours (D)	0.9630	0.9585	0.9647	0.9617	800 sample/s	400 ms/sample

Real World Deployment (RQ6): To further evaluate the scalability and practical applicability of TrafficLLM, we applied INT4 quantization to its dense version, TrafficLLM (D), and conducted experiments on an NVIDIA A100 GPU using the previously unseen NUDT-Mobile dataset [38]. This dataset contains 112.2 GB of traffic, comprising 1,157,245 flows across diverse protocols, including TCP, UDP, HTTP, TLSv1.2, SSLv2, and WebSocket. We report its performance, throughput, and latency results in Table IX. The results show that TrafficLLM achieves an F1 score of 0.9617, a throughput of 800 samples/s, and a latency of 400 ms/sample, indicating strong potential for deployment in real-world encrypted traffic classification scenarios.

VI. DISCUSSION AND FUTURE WORK

Task-Adaptive MoE Design: Although MoE-enabled SVD-LoRA improves performance on heterogeneous tasks, the expert

selection mechanism could be further refined using reinforcement learning or meta-learning strategies to dynamically adapt expert usage based on task complexity.

Enhanced Robustness and Interpretability: Future work will focus on improving the interpretability of model predictions, especially for security analysts. This includes techniques such as attention visualization, rationale extraction, and integration with threat intelligence sources.

Deployment at Scale: Further optimization of memory and computation costs through advanced quantization, pruning, and model distillation will be critical for large-scale deployment in production network environments.

VII. Conclusion

In this paper, we introduced `TrafficLLM`, a PEFT method designed for multi-task encrypted traffic analysis service. `TrafficLLM` addresses task heterogeneity through a universal multi-task prompt template and mitigates knowledge forgetting by integrating SVD-LoRA. By combining the strengths of the MoE approach with SVD-LoRA, `TrafficLLM` achieves efficient multi-task learning and reduces adaptation costs. The inclusion of task-aware gating functions further enhances efficiency by dynamically assigning weights to experts for optimal knowledge fusion. Our comprehensive experiments on seven datasets across five downstream tasks demonstrate that `TrafficLLM` significantly outperforms state-of-the-art models like NetGPT, ET-BERT, and ChatGLM (F) in terms of analysis performance and resource efficiency. Detailed evaluations of throughput, memory usage, and latency in the real world confirm the practical advantages of `TrafficLLM`.

References

[1] E. Papadogiannaki and S. Ioannidis, "A survey on encrypted network traffic analysis applications, techniques, and countermeasures," *ACM Comput. Surv.*, vol. 54, no. 6, pp. 1–35, 2021.

[2] D. Tang, S. Wang, B. Liu, W. Jin, and J. Zhang, "GASF-IPP: Detection and mitigation of LDoS attack in SDN," *IEEE Trans. Services Comput.*, vol. 16, no. 5, pp. 3373–3384, Sep./Oct. 2023.

[3] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "AppScanner: Automatic fingerprinting of smartphone apps from encrypted network traffic," in *Proc. IEEE Eur. Symp. Secur. Privacy*, 2016, pp. 439–454.

[4] I. Akbari et al., "A look behind the curtain: Traffic classification in an increasingly encrypted web," in *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 5, 2021, Art. no. 4.

[5] Q. Zhang, Y. Ma, J. Wang, and X. Li, "UDP traffic classification using most distinguished port," in *Proc. Asia-Pacific Netw. Operations Manag. Symp.*, 2014, pp. 1–4.

[6] M. Finsterbusch, C. Richter, E. Rocha, J.-A. Muller, and K. Hanssgen, "A survey of payload-based traffic classification approaches," *IEEE Commun. Surveys Tuts.*, vol. 16, no. 2, pp. 1135–1156, 2nd Quart., 2014.

[7] M. Crotti, M. Dusi, F. Gringoli, and L. Salgarelli, "Traffic classification through simple statistical fingerprinting," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 37, no. 1, pp. 5–16, 2007.

[8] X. Meng, C. Lin, Y. Wang, and Y. Zhang, "NetGPT: Generative pretrained transformer for network traffic," 2023, *arXiv:2304.09513*.

[9] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, 2022, pp. 633–642.

[10] X. Jiang, S. Liu, A. Gember-Jacobson, P. Schmitt, F. Bronzino, and N. Feamster, "Generative, high-fidelity network traces," in *Proc. ACM Workshop Hot Topics Netw.*, 2023, pp. 131–138.

[11] G. Aceto, D. Ciuonzo, A. Montieri, and A. Pescapé, "Mobile encrypted traffic classification using deep learning: Experimental evaluation, lessons learned, and challenges," *IEEE Trans. Netw. Service Manag.*, vol. 16, no. 2, pp. 445–458, Jun. 2019.

[12] T.-L. Huoh, Y. Luo, P. Li, and T. Zhang, "Flow-based encrypted network traffic classification with graph neural networks," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 2, pp. 1224–1237, Jun. 2023.

[13] P. Lin, K. Ye, Y. Hu, Y. Lin, and C.-Z. Xu, "A novel multimodal deep learning framework for encrypted traffic classification," *IEEE/ACM Trans. Netw.*, vol. 31, no. 3, pp. 1369–1384, Jun. 2023.

[14] K. Yang, L. Xu, Y. Xu, and J. Chao, "Encrypted application classification with convolutional neural network," in *Proc. IFIP Netw. Conf.*, 2020, pp. 499–503.

[15] C. V. Wright, F. Monrose, and G. M. Masson, "On inferring application protocol behaviors in encrypted network traffic," *J. Mach. Learn. Res.*, vol. 7, no. 12, pp. 2745–2769, 2006.

[16] M. Conti, L. V. Mancini, R. Spolaor, and N. V. Verde, "Analyzing android encrypted network traffic to identify user actions," *IEEE Trans. Inf. Forensics Security*, vol. 11, no. 1, pp. 114–125, Jan. 2016.

[17] O. Sener and V. Koltun, "Multi-task learning as multi-objective optimization," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 525–536.

[18] J. Dai, X. Xu, H. Gao, and F. Xiao, "CMFTC: Cross modality fusion efficient multitask encrypt traffic classification for efficient management of IIoT," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 6, pp. 3989–4009, Nov./Dec. 2023.

[19] D. Wu et al., "Large language model adaptation for networking," in *Proc. ACM Conf. Special Int. Group Data Commun.*, 2024, pp. 661–678.

[20] B. Zheng et al., "Adapting large language models by integrating collaborative semantics for recommendation," in *Proc. IEEE 40th Int. Conf. Data Eng.*, 2024, pp. 1435–1448.

[21] A. Radford et al., "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, 2019, Art. no. 9.

[22] A. Vaswani et al., "Attention is all you need," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 6000–6010.

[23] C. H. Song, J. Wu, C. Washington, B. M. Sadler, W.-L. Chao, and Y. Su, "LLM-planner: Few-shot grounded planning for embodied agents with large language models," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 2986–2997.

[24] W. Wang et al., "VisionLLM: Large language model is also an open-ended decoder for vision-centric tasks," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2024, pp. 61501–61513.

[25] X. He, X. Bresson, T. Laurent, A. Perold, Y. LeCun, and B. Hooi, "Harnessing explanations: LLM-to-LM interpreter for enhanced text-attributed graph representation learning," in *Proc. Int. Conf. Learn. Representations*, 2023.

[26] R. Zhao, X. Deng, Z. Yan, J. Ma, Z. Xue, and Y. Wang, "MT-FlowFormer: A semi-supervised flow transformer for encrypted traffic classification," in *Proc. ACM Int. Conf. Knowl. Discov. Data Mining*, 2022, pp. 2576–2584.

[27] T. Zheng, J. Shao, J. Dai, S. Jiang, X. Chen, and C. Shen, "RESTLess: Enhancing state-of-the-art rest API fuzzing with LLMs in cloud service computing," *IEEE Trans. Services Comput.*, vol. 17, no. 6, pp. 4225–4238, Nov./Dec. 2024.

[28] G. Zhou, X. Guo, Z. Liu, T. Li, Q. Li, and K. Xu, "TrafficFormer: An efficient pre-trained model for traffic data," in *Proc. IEEE Symp. Secur. Privacy*, 2025, pp. 1844–1860.

[29] R. Zhao et al., "Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 5420–5427.

[30] S. K. Mani et al., "Enhancing network management using code generated by large language models," in *Proc. ACM Workshop Hot Topics Netw.*, 2023, pp. 196–204.

[31] B. Piggott et al., "Net-GPT: A LLM-empowered man-in-the-middle chatbot for unmanned aerial vehicle," in *Proc. IEEE/ACM Symp. Edge Comput.*, 2023, pp. 287–293.

[32] S. Zhang, D. Fu, Z. Zhang, B. Yu, and P. Cai, "TrafficGPT: Viewing, processing and interacting with traffic foundation models," 2023, *arXiv:2309.06719*.

[33] H. Y. He, Z. G. Yang, and X. N. Chen, "PERT: Payload encoding representation from transformer for encrypted traffic classification," in *Proc. ITU Kaleidoscope: Industry-Driven Digit. Transformation*, 2020, pp. 1–8.

[34] Y. Zhai et al., "Investigating the catastrophic forgetting in multimodal large language model fine-tuning," in *Proc. 1st Conf. Parsimony Learn.*, 2024, pp. 202–227.

[35] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-Net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE Conf. Comput. Commun.*, 2019, pp. 1171–1179.
[36] H. Zhang et al., "TFE-GNN: A temporal fusion encoder using graph neural networks for fine-grained encrypted traffic classification," in *Proc. ACM Web Conf.*, 2023, pp. 2066–2075.
[37] G. Team et al., "ChatGLM: A family of large language models from GLM-130B to GLM-4 all tools," 2024, *arXiv:2406.12793*.
[38] S. Zhao, S. Chen, F. Wang, Z. Wei, J. Zhong, and J. Liang, "NUDT mobile traffic dataset," (n.d.). Accessed: Apr. 05, 2025. [Online]. Available: https://github.com/Abby-ZS/NUDT_MobileTraffic
[39] C. Cui et al., "A survey on multimodal large language models for autonomous driving," in *Proc. Winter Conf. Appl. Comput. Vis.*, 2024, pp. 958–979.
[40] W. Feng, C. Hao, Y. Zhang, Y. Han, and H. Wang, "Mixture-of-LoRAs: An efficient multitask tuning for large language models," 2024, *arXiv:2403.03432*.
[41] S. Chen, Y. Zhang, and Q. Yang, "Multi-task learning in natural language processing: An overview," *ACM Comput. Surv.*, vol. 56, 2021, Art. no. 295.
[42] A. Sarabi, T. Yin, and M. Liu, "An LLM-based framework for fingerprinting internet-connected devices," in *Proc. Internet Meas. Conf.*, 2023, pp. 478–484.
[43] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw.*, 2017, pp. 712–717.
[44] K. Bostrom and G. Durrett, "Byte pair encoding is suboptimal for language model pretraining," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2020, pp. 4617–4624.
[45] L. Huang et al., "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Trans. Inf. Syst.*, vol. 43, no. 2, pp. 1–55, 2025.
[46] Q. Zhang et al., "Adaptive budget allocation for parameter-efficient fine-tuning," in *Proc. Int. Conf. Learn. Representations*, 2024.
[47] W. Cai, J. Jiang, F. Wang, J. Tang, S. Kim, and J. Huang, "A survey on mixture of experts," 2024, *arXiv:2407.06204*.
[48] H. Nguyen, T. Nguyen, and N. Ho, "Demystifying softmax gating function in Gaussian mixture of experts," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 4624–4652.
[49] Q. Liu et al., "When MOE meets LLMs: Parameter efficient fine-tuning for multi-task medical applications," in *Proc. 47th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2024, pp. 1104–1114.
[50] T. Van Ede et al., "Flowprint: Semi-supervised mobile-app fingerprinting on encrypted network traffic," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020.
[51] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related," in *Proc. 2nd Int. Conf. Inf. Syst. Secur. Privacy*, 2016, pp. 407–414.
[52] A. H. Lashkari, G. D. Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of tor traffic using time based features," in *Proc. Int. Conf. Inf. Syst. Secur. Privacy*, 2017, pp. 253–262.
[53] A. Liu et al., "DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model," 2024, *arXiv:2405.04434*.
[54] M. Boratko, X. Li, T. O'Gorman, R. Das, D. Le, and A. Mccallum, "ProtoQA: A question answering dataset for prototypical common-sense reasoning," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2020, pp. 1122–1136.
[55] S. Dadkhah, H. Mahdikhani, P. K. Danso, A. Zohourian, K. A. Truong, and A. A. Ghorbani, "Towards the development of a realistic multidimensional IoT profiling dataset," in *Proc. 19th Annu. Int. Conf. Privacy, Secur. Trust*, 2022, pp. 1–11.
[56] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learn. Representations*, 2023.

Yi Liu is currently working toward the PhD degree with the Department of Computer Science, City University of Hong Kong. His research interests include cloud computing security, network security, and privacy-preserving machine learning.

Xiang Zheng received the PhD degree from the Department of Computer Science, City University of Hong Kong, in 2024, under the guidance of Prof. Cong Wang. He is a postdoctoral fellow with the Department of Computer Science, City University of Hong Kong. His research interests include intersection of reinforcement learning, trustworthy AI, generative AI, and robot learning.

Chengjun Cai (Member, IEEE) received the PhD degree in computer science from the City University of Hong Kong, in 2021. He is currently an associate professor with the City University of Hong Kong (Dongguan). His research interests included applied cryptography, data security and privacy, and blockchain. He is a member of the ACM.

Xingliang Yuan (Senior Member, IEEE) is an associate professor with the School of Computing and Information Systems, University of Melbourne. Prior to that, he was a faculty member with Monash University from 2017 to 2024. He has a keen interest in designing systems, protocols to address real-world privacy, and security challenges. His current research focuses on data security and privacy, secure networked system, and trustworthy machine learning. His research has been supported by Australian Research Council, CSIRO, Australian Department of Home Affairs, Australian Department of Health and Aged Care, and the Oceania Cyber Security Centre. His work has been published in major venues of computer security and systems, such as CCS, S&P, USENIX Security, NDSS, *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Information Forensics and Security*, etc. He is a sole recipient of the Dean's Award for Excellence in Research by an Early Career Researcher (2020) at Monash. He is a co-recipient of the best paper award in the European Symposium on Research in Computer Security (ESORICS) 2021. He is currently on the editorial board of *IEEE Transactions on Dependable and Secure Computing (TDSC)* and *IEEE Transactions on Service Computing (TSC)*. He served as a track co-chair of ICDCS'24, WISE'24, MSN'24, and a program co-chair of Lamps@CCS'24, SecTL@AsiaCCS'23, and NSS'22.

Cong Wang (Fellow, IEEE) is currently a chair professor with the Department of Computer Science, City University of Hong Kong. He is also with the City University of Hong Kong Shenzhen Research Institute. His research interests include data and network security, blockchain and decentralized applications, and privacy-enhancing technologies. He is a member of ACM. He has been one of the founding member of the Young Academy of Sciences of Hong Kong since 2017. He was conferred the RGC Research Fellow, in 2021. He received the Outstanding Researcher Award (junior faculty), in 2019, the Outstanding Supervisor Award, in 2017, and the President's Awards in 2016 and 2019, all from the City University of Hong Kong. He was a co-recipient of the Best Paper Award of IEEE ICDCS 2020, ICPADS 2018, MSN 2015, the Best Student Paper Award of IEEE ICDCS 2017, and the IEEE INFOCOM Test of Time Paper Award in 2020. His research has been supported by multiple government research fund agencies, including the National Natural Science Foundation of China, the Hong Kong Research Grants Council, and the Hong Kong Innovation and Technology Commission. He served as the TPC co-chair for a number of IEEE conferences and workshops. He has served as an editor-in-chief for *IEEE Transactions on Dependable and Secure Computing*, and editor for *IEEE Transactions on Services Computing*, *IEEE Internet of Things Journal*, *IEEE Networking Letters*, and the *Journal of Blockchain Research*.